Universität Konstanz

Applied Natural Language Processing (NLP) for Online Harm Detection

Jun.-Prof. Dr. Miriam Schirmer

Konstanz, 05.11.2026

Outline for Today

Session 3: Supervised Classification

1. Recap
2. Readings: J&M ch. 4 and “Garbage in, garbage out”
3. Text classification
4. Text as features: bag of words
5. Logistic regression (intuition)
6. Evaluation: precision, recall, F1
7. Annotation and agreement
8. Your turn: annotation exercise (with an AI annotator)

Last week: lexicons

What lexicons can do

- Count words from a list (e.g., a list of insults or slurs)
- Cheap, fast, transparent

What they cannot do

- **Context:** “I’ll kill you” (threat) vs. “this song kills” (praise)
- **New or coded words:** new slang, misspellings, coded terms are missing from the list
- **Implicit harm:** “Women just aren’t cut out for leadership roles” contains no bad word
- **Counter-speech:** quoting a slur to criticize it is counted as hate

Let’s talk about the readings!

Let’s talk about the readings

- **Jurafsky & Martin, ch. 4:** one thing you learned or found surprising about how a classifier learns from labelled examples
- **Vidgen & Derczynski (2020):** what makes a good training dataset for abusive language, and what goes wrong in many existing ones?

Vidgen & Derczynski: data

	Finding
Platforms	39 of 63 datasets use Twitter (36 only Twitter); Facebook 6; gaming platforms only 2
Languages	English most common (25 datasets); then Arabic (6), Italian (5); many languages missing

Vidgen & Derczynski: labels

	Finding
Annotators	Experts or trained workers 44%, crowdworkers 33%, mixed 11%, synthetic 6%, moderators 5%
Who are they?	No information on the annotators for 27 datasets, very limited for 24, detailed for only 12
Guidelines	43% share no guidelines, 32% only a short summary, 25% detailed guidelines
Agreement	Often low, especially for detailed (multi-class) categories
Definitions	Categories and definitions differ a lot between datasets

Vidgen & Derczynski: balance and prevalence

	Finding
Balance	On average 36.7% of posts are abusive (range 1% to 100%)
Reality	**On real platforms probably well under 1%: most datasets are not representative**

What do Vidgen & Derczynski recommend?

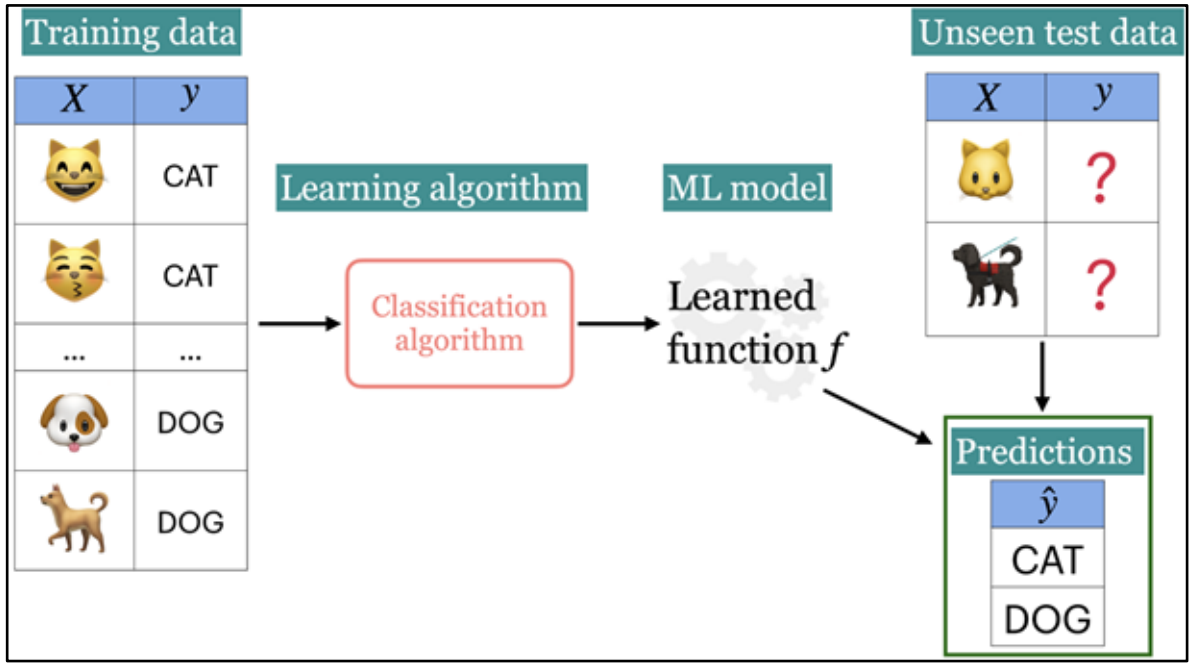
Vidgen & Derczynski: recommendations

- Build datasets for a **specific, well-defined task**, not a “general purpose” one
- Look beyond Twitter: other **platforms** and **languages**
- Use trained annotators and **clear guidelines with examples of edge cases**; refine them together with the annotators
- Document the data with **data statements** (who wrote it, who labelled it, how it was sampled)
- Share **all annotations**, not only the final label
- For practitioners: check whether a dataset fits your task, platform and class balance; avoid relying on **word lists**

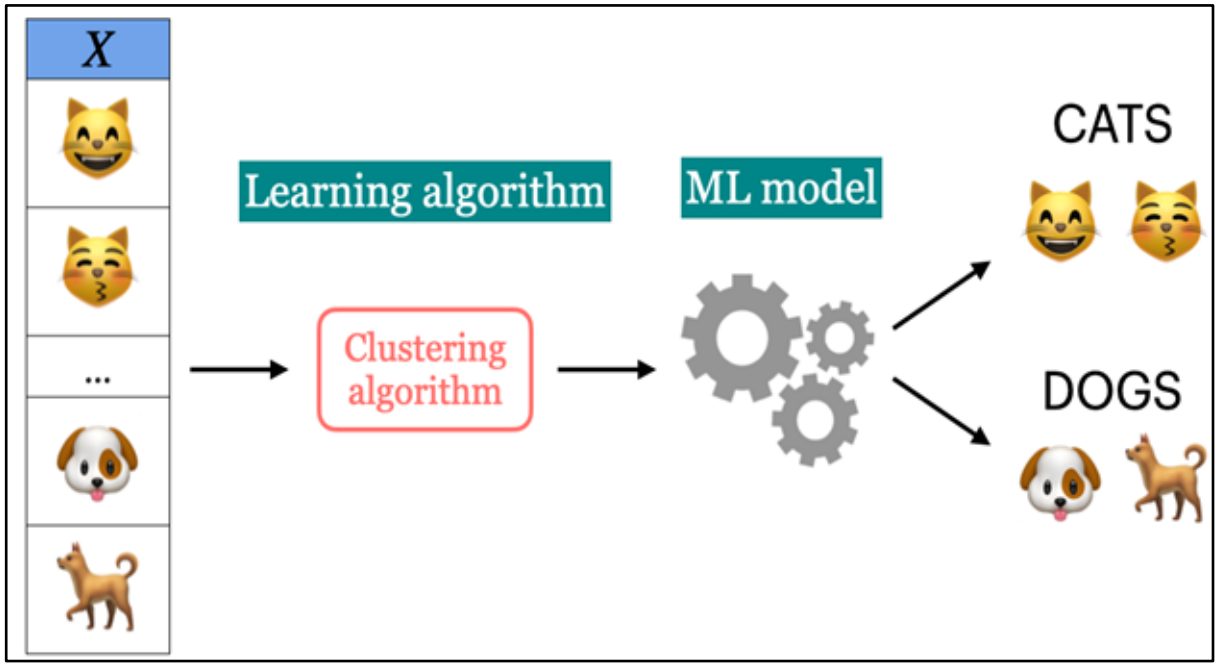
Can a computer learn what counts as “abusive” from examples?

Text classification

Supervised Learning



Unsupervised Learning



Supervised (text classification): learn patterns from labeled data

Unsupervised (topic modeling, clustering): learn patterns from unlabeled data

https://ubc-cs.github.io/cpsc330-2024W2/lectures/notes/15_K-Means.html

Text classification is everywhere

- Topic or Genre Categorization
- Sentiment Analysis
- Author Identification
- Language Detection
- Spam Filter
- …..

- What other applications of text classification can you think of?

Classification in online harm research

Typical tasks

- **Hate speech / abusive language:** yes/no, or type (e.g. racist, sexist), or target
- **Toxicity:** probability that a comment is “rude, disrespectful or unreasonable” (e.g. Perspective API)
- **Misinformation:** is a claim check-worthy? Does a source support or refute it?
- **Cyberbullying, incivility, extremist content, threats**

Why classification?

- Millions of posts per day: platforms use classifiers for **content moderation**
- Researchers use them to **measure** harm at scale (e.g. how much hate speech after an event?)
- Classifier errors are not just numbers: they decide **what stays online**

Text Classification Definition

Input:

- A text document d
- A fixed set of classes $Y = \{y_1, y_2, \ldots, y_j\}$

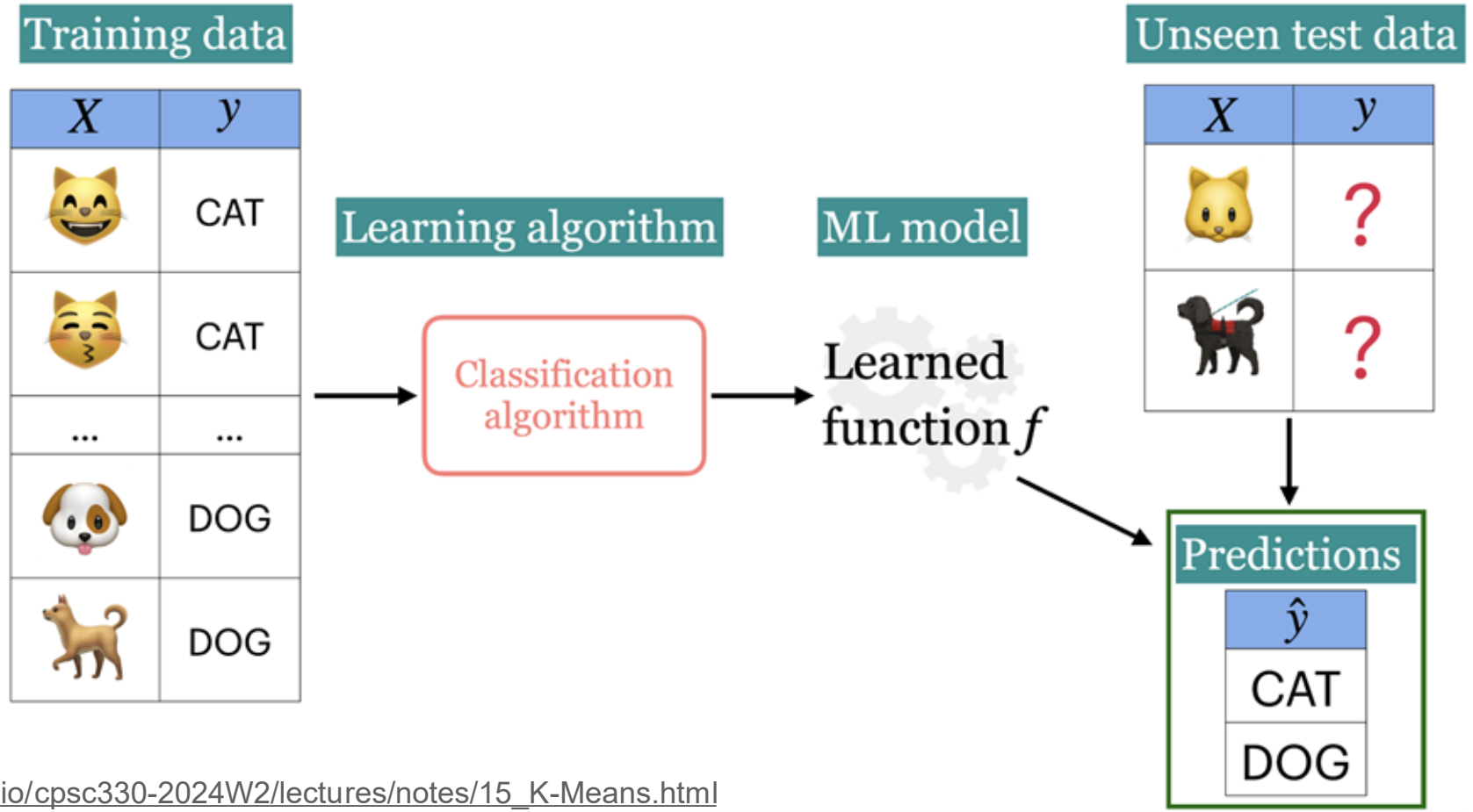
Output:

- A predicted class $\hat{y} \in Y$

d	*I love this movie…*
Y	{ positive, negative }
$\hat{y}$	positive

The hat (circumflex) notation ŷ refers to an estimated or predicted value.

Our goal is to figure out what function *f* gives the best predictions



https://ubc-cs.github.io/cpsc330-2024W2/lectures/notes/15_K-Means.html

There are many kinds of classifiers

- **Logistic Regression**
- Naive Bayes
- Neural Networks
- *K*-nearest neighbors
- Support Vector Machines
- Random Forest
- Fine-tuned LLMs as classifiers (BERT)
- LLM prompting

Machine Learning for NLP

Two components



Classification models



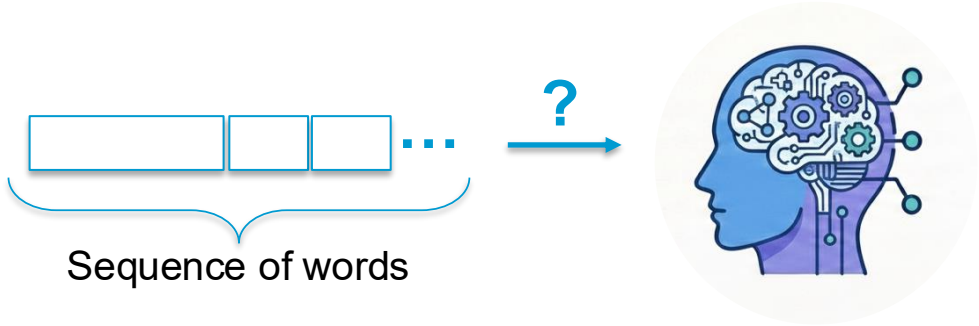
Text representations

Machine learning models do not work *directly* on text!

Text as features

Discrete representations for text

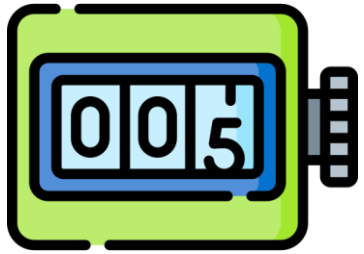
Problem: **ML models work on vectors of numbers… how do we represent text that way?**



Discrete representations for text

Idea: count words!

If a text is defined by the words it contains, then **counting their occurrences** seems very natural.



Bag of words representations

Bag of words (BoW)

Setup:



Word w



Document d



Corpus C



Vocabulary V

Bag of words (BoW)

Algorithm (BoW representation)

1. **Build vocabulary** $V \longrightarrow V = \{w_1, w_2, \ldots, w_{|V|}\}$ 

2. **For each document** d_i **count how many times each word appears in it.** 

$$d_i \longrightarrow \left[c_1^{(i)}, \quad c_2^{(i)}, \quad \ldots \quad , \quad c_{|V|}^{(i)}\right] \longrightarrow \text{«Term frequencies» (TF) vector for document } \quad i$$

Number of occurrences of word w_1

Number of occurrences of word w_2

Number of occurrences of word $w_{|V|}$

Bag of words (BoW)

Algorithm (BoW representation)

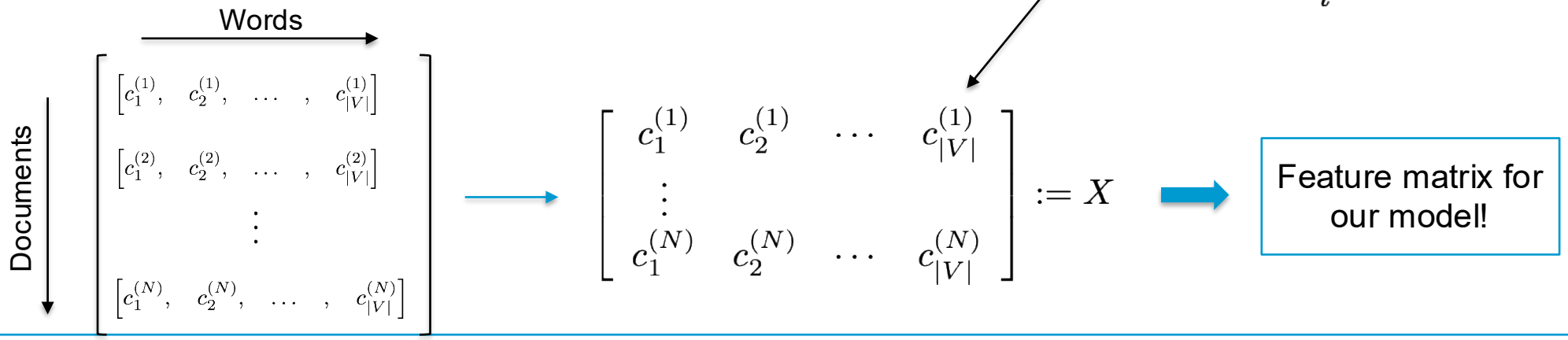
1. **Build vocabulary** $V \longrightarrow V = \{w_1, w_2, \ldots, w_{|V|}\}$ 

2. **For each document** d_i **count how many times each word appears in it** 

$$d_i \longrightarrow \left[c_1^{(i)}, \quad c_2^{(i)}, \quad \ldots \quad , \quad c_{|V|}^{(i)}\right] \longrightarrow \text{«Term frequencies» (TF) vector for document } \quad i$$

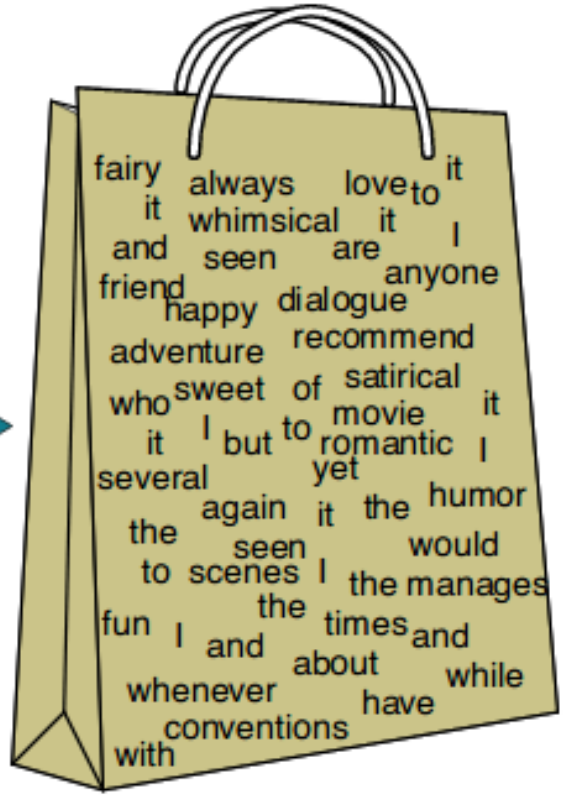
3. **Collect all the counts for each document in a single matrix.**

$c_j^{(i)}$ = frequency of word w_j in document d_i



Bag of words (BoW)

I love this movie! It's sweet, but with satirical humor. The dialogue is great and the adventure scenes are fun... It manages to be whimsical and romantic while laughing at the conventions of the fairy tale genre. I would recommend it to just about anyone. I've seen it several times, and I'm always happy to see it again whenever I have a friend who hasn't seen it yet!

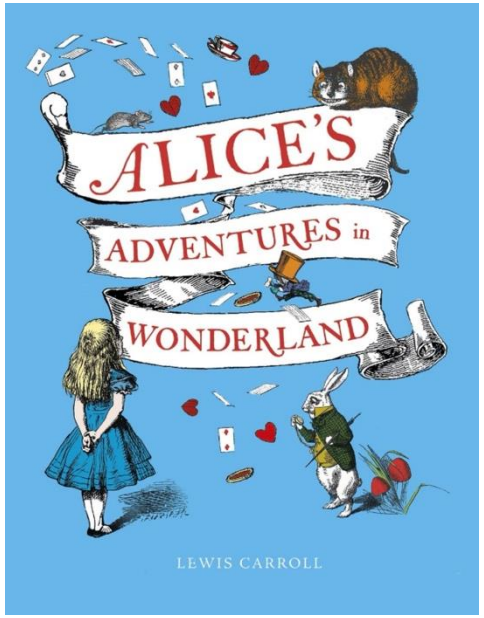


it	6
I	5
the	4
to	3
and	3
seen	2
yet	1
would	1
whimsical	1
times	1
sweet	1
satirical	1
adventure	1
genre	1
fairy	1
humor	1
have	1
great	1
…	…

https://medium.com/@vamshiprakash001/an-introduction-to-bag-of-words-bow-c32a65293ccc

Bag of words (BoW)

Problem: if we count word frequencies in any document, we immediately realize that most of the words are not important for any classification task – they just naturally occur more frequently given how language is constructed.



	word	counts
0	the	570
1	and	300
2	a	265
3	to	244
4	she	198
5	of	193
6	Alice	166
7	was	163
8	in	155
9	it	153
10	said	143
11	I	124

All generic words, apart from «Alice»

We need a way to give less importance to words that appear almost always

TF-IDF

Idea: use the whole corpus to reweight each word's (term's) frequency by its document frequency, i.e. by how often a word appears across documents.

High document frequency → Very common words, probably not important → Downweight counts.
Low(er) document frequency → Less common words, maybe more important → Upweight counts.

Formula:

Term frequency (counts)

«Inverse document frequency»

$$\text{TF-IDF}(w_j, d_i) = \text{TF}(w_j, d_i) \cdot \text{IDF}(w_j, C)$$

$$= c_j^{(i)} \cdot \log\left(\frac{|C|}{d_f(w_j, C)}\right)$$

Corpus size

Using the log here allows the up-weighting not to be too steep

«Document frequency» = number of documents in which word w appears

Normalization!

TF vs. TF-IDF: an example

Corpus: N = 4 posts

- Post 1: “you are so stupid”
- Post 2: “you are so right”
- Post 3: “you are an idiot”
- Post 4: “thank you so much”

IDF = log(N / DF)

- TF = how often the word appears in that one post
- DF = in how many posts the word appears
- “you” is in every post: log(4/4) = 0
- “stupid” is only in post 1: log(4/1) = 0.60

With counts (TF), all four words of post 1 count the same. With TF-IDF, “stupid” gets the highest weight and “you” drops out.

Post 1	TF	DF	IDF	TF-IDF
“you”	1	4	0.00	0.00
“are”	1	3	0.12	0.12
“so”	1	3	0.12	0.12
“stupid”	1	1	0.60	**0.60**

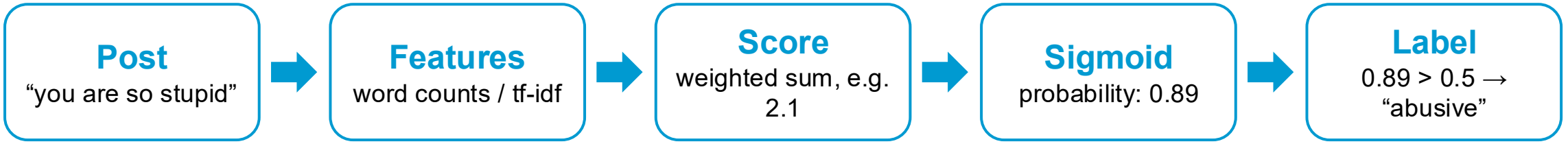
The higher the weight, the more distinctive this word is for this post

Features in logistic regression

- Features often include *n-gram* counts
  - Usually 1-2 gram and tf-idf normalized
- Frequency of words from lexicons (e.g. LIWC “positive emotion” words)
- Stylistic features (e.g. length, capitalization, punctuation)
- …
- Our model learns *weights* that tell us how important each feature is.
  - x_i = review contains ***awesome***: $w_i = +10$
  - x_j = review contains ***horrible***: $w_j = -10$
  - x_k = review contains ***mediocre***: $w_k = -2$

Logistic regression in NLP

Why use logistic regression?



Why the logistic function?

- The score can be any number (−7, 0.3, 12…), but we want a probability
- The sigmoid maps every score to a value between 0 and 1: score 0 → 0.5, high scores → close to 1, low scores → close to 0
- A threshold (usually 0.5) turns the probability into a class

Why logistic regression?

- Common baseline algorithm for classification tasks
- Other classifiers (Naive Bayes, SVMs, random forests) would also work on the same features
- Logistic regression gives probabilities and weights we can read
- Neural classifiers (e.g. BERT) end in the same sigmoid/softmax step

Logistic Regression

Example

Important analytic tool in natural and social sciences

E.g., classify hostile framing with BERT, then use logistic regression to analyze the relationship between political ideology and hostile framing.

Components of a probabilistic machine learning classifier

1. A feature representation of the input text. For each input observation $x^{(i)}$, we extract a vector of features $[x_1, x_2, \ldots, x_n]$
2. A classification function **f** that takes a feature vector as input and computes the estimated class $\hat{y}$ based on estimated probabilities.
3. An objective function to learn *weights* in **f**
4. An algorithm for optimizing the objective function

We'll focus on logistic regression, but the core concepts apply across many types of models, including BERT and LLMs.

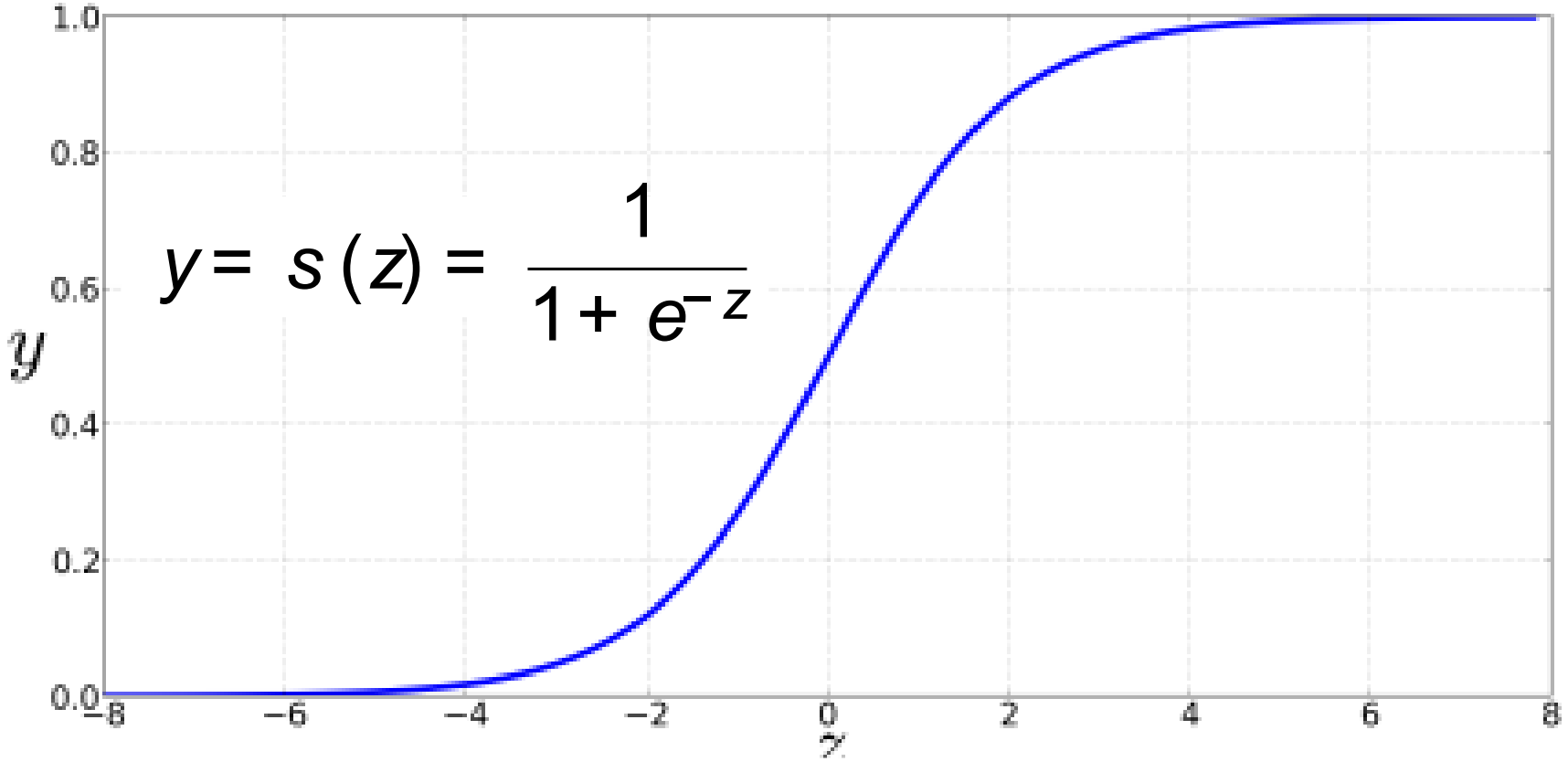
Logistic regression without formulas

1. Turn each post into **features** (which words appear, how often)
2. Each feature has a **weight**: positive = pushes towards “abusive”, negative = pushes away
3. Add up the weights of the features in a post → a **score**
4. Reshape the score into a **probability** between 0 and 1 (the sigmoid)
5. Probability above 0.5 → “abusive”, otherwise “not abusive”

Where do the weights come from?

- From the **training data**: the model adjusts the weights step by step until its predictions match the human labels as well as possible (“gradient descent”)

The sigmoid or logistic function



What could a model learn? (made-up example)

Feature (word)	Weight	Why?
idiot	+2.1	insult, mostly in abusive posts
clown	+1.4	insult, but also used jokingly among friends
thanks	−1.3	mostly in friendly posts
football	+0.9	⚠ many abusive posts in the training data were about football
women	+0.7	⚠ identity term: often mentioned in sexist posts, but also everywhere else
the	0.0	appears everywhere, no information

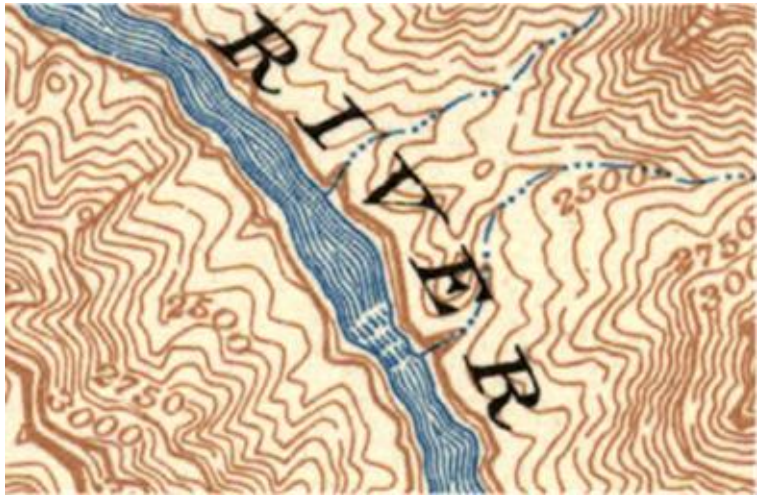
Where do the weights come from?

- In our training data, we know the correct label *y* (0 or 1) for each *x*
- We want to set *w* and *b* to **minimize the distance** between our estimate $\hat{y}$ and the true *y*
- We estimate this distance with a **loss function** (a.k.a. cost function)
  - cross-entropy loss
- We need an **optimization algorithm** to update w and b to minimize the loss.
  - stochastic gradient descent

Note: we're focusing on binary classification for simplicity. Multi-class (multinomial) logistic regression has the same logic but slightly different math

Our goal: minimize the loss

- We want to find the weights that minimize the cross-entropy loss, averaged over all examples
- We update the weights using **gradient descent**
- Intuition: how do I get to the bottom of this canyon?
  - Look around yourself
  - Find the direction of the steepest slope down
  - Go that way



Evaluation

Generalizing to data unseen in training

- What if our model is just learning the patterns of our specific training data?
- Standard practice is to evaluate models on a **held-out test set**

Generalizing to data unseen in training

- **Training Set:** used to learn model parameters
  - Typically 70-80% of your labeled data
- **Validation/Development Set:** used to learn hyperparameters, debug, choose best model instance
  - Typically 10-15% of your labeled data
  - People often forego validation set and just do train/test split
- **Test Set:** used to do final evaluation of model performance
  - Typically 10-15% of your labeled data, 20-30% if you don't have a validation set
- You can alternatively do **K-fold cross-validation**
  - Split data into K equal-sized "folds" or subsets (usually 5 or 10)
  - Train on K - 1 folds and test on the remaining fold, average performance across folds

Confusion Matrix

In the binary classification case we can have:

		Predicted condition	
		Predicted positive	Predicted negative
Actual condition	Real positive (P)	**True positive (TP)**	**False negative (FN)**
	Real negative (N)	**False positive (FP)**	**True negative (TN)**

Four possible cases, based on the actual VS the predicted condition.

→ **Metrics**

Precision and Recall

Out of the texts predicted as spam, what fraction are actually spam?

$$Precision = \frac{TP}{TP + FP}$$

Out of the documents that are actually spam, what fraction are predicted as such?

$$Recall = \frac{TP}{TP + FN}$$

		Predicted condition	
		Predicted positive	Predicted negative
	Real positive (P)	**True positive (TP)**	**False negative (FN)**
	Real negative (N)	**False positive(FP)**	**True negative (TN)**

Precision or recall? It depends on who pays for the errors

A hate speech classifier used for content moderation

- **Low recall** (misses hate speech): harmful posts stay online, targets are left alone
- **Low precision** (flags harmless posts): over-removal, e.g. of counter-speech, reclaimed words or minority dialects

Classification metrics

Model evaluation uses the typical classification metrics for supervised learning, evaluated on a labelled test set:

Metric	Definition	Meaning
Accuracy	$A = \frac{\mathrm{TP} + \mathrm{TN}}{\mathrm{TP} + \mathrm{TN} + \mathrm{FP} + \mathrm{FN}}$	Overall rate of correct predictions
Precision	$P = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FP}}$	Fraction of samples predicted positive that actually are
Recall	$R = \frac{\mathrm{TP}}{\mathrm{TP} + \mathrm{FN}}$	Fraction of actually positive samples that are predicted positive
F1 score	$F_1 = \frac{2}{\frac{1}{P} + \frac{1}{R}} = \frac{2 \cdot P \cdot R}{P + R}$	Combination of P and R less sensitive to class imbalance

Why don't we use accuracy?

Accuracy doesn't work well when we're dealing with uncommon or imbalanced classes

Suppose we look at 1,000,000 social media posts to find Pie-lovers (or haters)

- 100 of them talk about our pie
- 999,900 are posts about something unrelated

We create a simple classifier that always says: **“nothing is pie”**

Example from J&M

Why don't we use accuracy?

Accuracy of our "nothing is pie" classifier

- 999,900 true negatives and 100 false negatives
- Accuracy is 999,900/1,000,000 = **99.99%**!
- But useless at finding pie-lovers (or haters)!!
- Which was our goal!

Accuracy doesn't work well for unbalanced classes

What harm can wrong classification cause?

What harm can wrong classification cause?

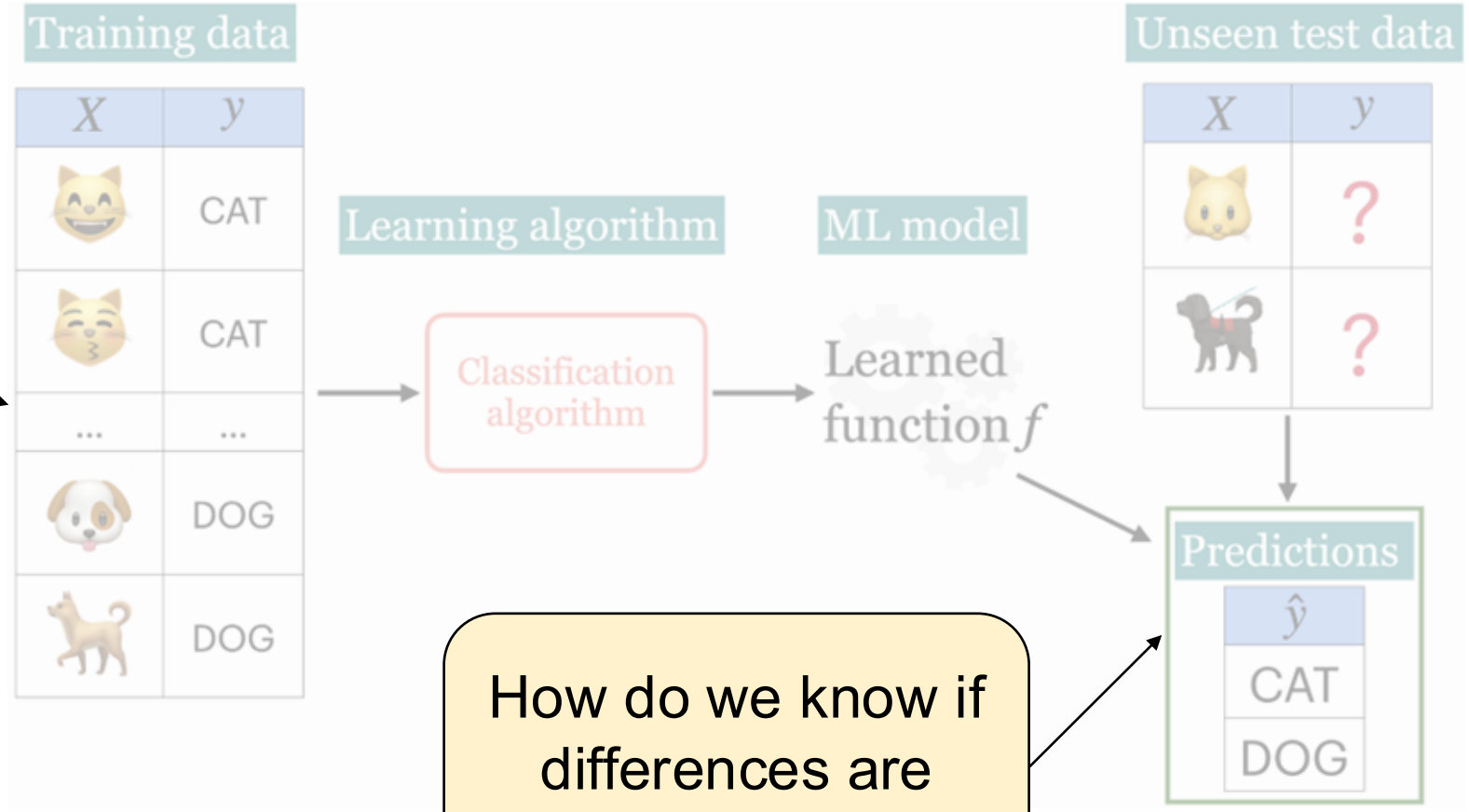
- **Representational harm:** systems demean groups, e.g. 200 sentiment systems gave sentences with African American names more negative scores (Kiritchenko & Mohammad 2018)
- **Censorship:** toxicity classifiers flag harmless sentences that only mention identities (e.g. “blind”, “gay”), so these groups’ posts become less visible
- **Performance disparities:** classifiers work worse for many languages and for dialects such as African American English

Source: Jurafsky & Martin, ch. 4 (harms in classification)

Annotation and agreement

Recap of Text Classification

But where do these labels come from?



How do we know if differences are meaningful?

What makes annotation hard?

What makes annotation hard?

- Socially-oriented annotation can be highly subjective and ground truth may not be well-defined
  - Sentiment, emotions
  - Politeness
  - Offensiveness, incivility
- Different people might interpret the same text in different ways
- Some constructs might have ground truth (based on specific definitions) but require a lot of training and expertise to correctly identify
  - Violation of democratic norms
  - Populist or authoritarian rhetoric

Two ways to annotate subjective tasks

Prescriptive annotation

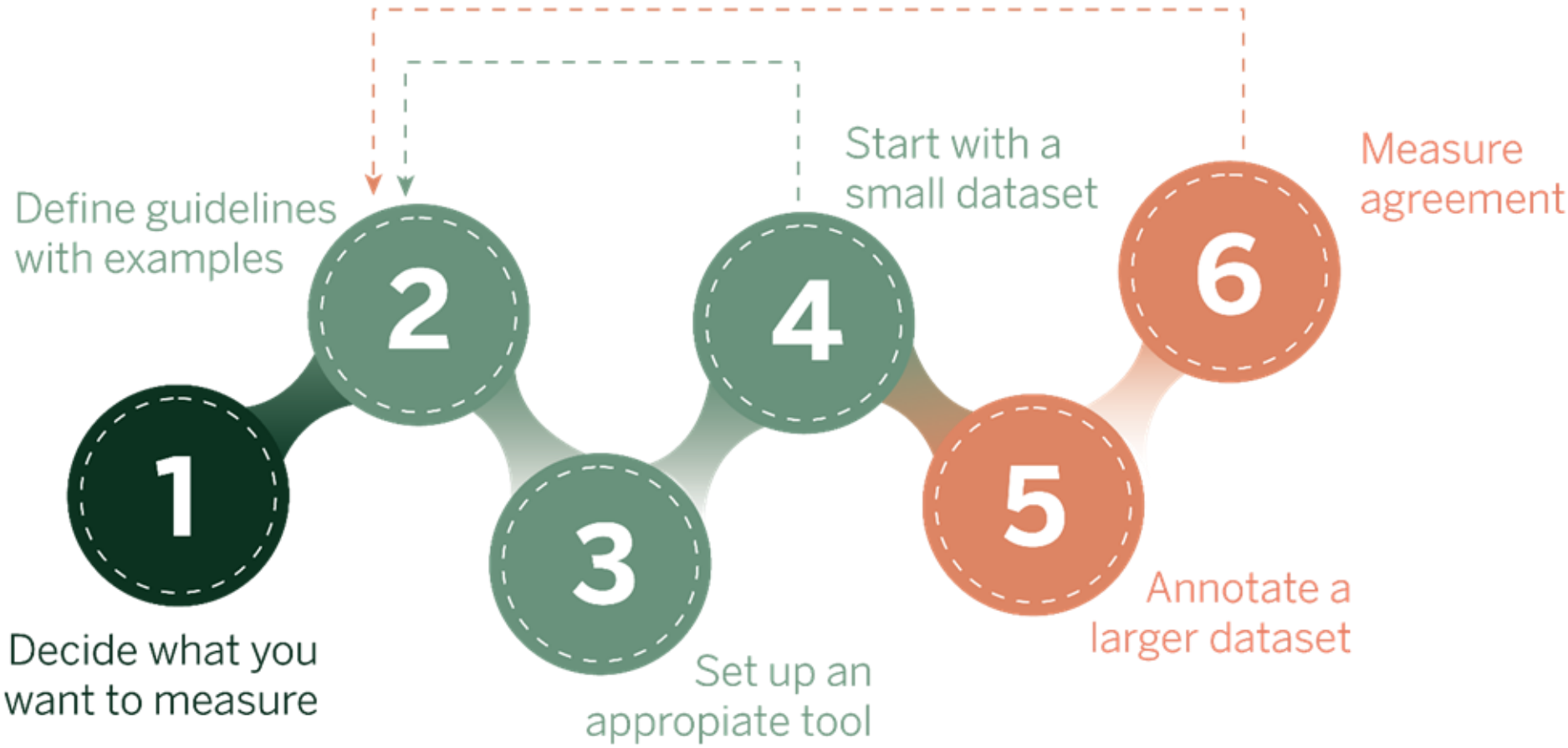
- Annotators follow **detailed guidelines** that define one “correct” reading
- Goal: consistent labels, high agreement, one belief encoded in the data
- E.g. a platform applying its community rules

Descriptive annotation

- Annotators are encouraged to give **their own view**
- Goal: capture how different people perceive a text; disagreement is information
- E.g. how offensive do members of the targeted group find this post?

Röttger, P., Vidgen, B., Hovy, D. & Pierrehumbert, J. (2022). Two Contrasting Data Annotation Paradigms for Subjective NLP Tasks. NAACL.

Annotation process at a very high level



https://www.bbvaaifactory.com/wp-content/uploads/Mesa-de-trabajo-4humanannotbn.png

How much data do I need to annotate

- Usually, as much as you can within your time and money budget
- You can also measure intermediate model performance with different amounts of annotated training data.
  - Once model performance stops improving as a function of training data size, you could probably stop.

Measure agreement

- How can we tell if annotations are reliable?
- Standard metric: inter-annotator agreement
  - Aka intercoder reliability, inter-rater reliability
- Each data point is annotated by multiple people
- Possible sources of disagreement
  - Maybe some instances are particularly challenging
  - Task is too hard or categories are poorly defined
  - Annotators weren't qualified or properly trained
  - Maybe there's inherently a lot of subjectivity in the task



Agreement without formulas

Percent agreement

- In how many of the posts did two annotators give the same label? E.g. 8 of 10 = 80%

The problem: agreement by chance

- If 9 of 10 posts are clearly harmless, two annotators who always answer “no” agree 90% of the time, without even reading

Kappa (Cohen’s, Fleiss’) and Krippendorff’s alpha

- Ask: **how much better than chance** is the agreement?
- **1** = perfect agreement, **0** = no better than chance, **below 0** = worse than chance
- We let the computer calculate it (spreadsheet, Python)

How to read kappa (a rough guide)

Scale from Landis & Koch (1977). With only 10 items, kappa jumps around a lot: treat it as a rough signal.

Kappa	Agreement	What it means for your data
below 0	poor	worse than chance: something is wrong
0.01–0.20	slight	labels are hardly reliable
0.21–0.40	fair	common for subjective tasks such as hate speech
0.41–0.60	moderate	usable, but check the guidelines (often seen as acceptable for subjective tasks)
0.61–0.80	substantial	good
0.81–1.00	almost perfect	very good (or a very easy task)

Cohen's Kappa

		Annotator 1		
		Not offensive	**Offensive**	**Sum**
Annotator 2	**Not offensive**	**147**	3	150
	Offensive	10	**62**	72
	Sum	157	65	222

- Chance agreement accounts for number of times each annotator selects each category
- Here, Cohen's kappa = 0.863
- Fleiss' Kappa extends this to more than two raters
- Pros: interpretability, easy to compute
- Cons: limited to categorical data, can't handle missing data

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

p_o = percent agreement

p_e = chance agreement

0 → agreement is random chance

- → agreement is worse than random

Cohen’s kappa step by step

		Annotator 1		
		Not offensive	**Offensive**	**Sum**
Annotator 2	**Not offensive**	**147**	3	150
	Offensive	10	**62**	72
	Sum	157	65	222

Rows = Annotator 2, row sums 150 and 72
Columns = Annotator 1, column sums 157 and 65
Total = 222 posts

1. Observed agreement p_o (yellow cells)

- p_o = (147 + 62) / 222 = **0.941**

2. Chance agreement p_e (from the sums)

- Not offensive: 150/222 × 157/222 = 0.478
- Offensive: 72/222 × 65/222 = 0.095
- p_e = 0.478 + 0.095 = **0.573**

3. Kappa: $\kappa = (p_o - p_e) / (1 - p_e)$

- κ = (0.941 − 0.573) / (1 − 0.573)
- = 0.368 / 0.427 = **0.86**

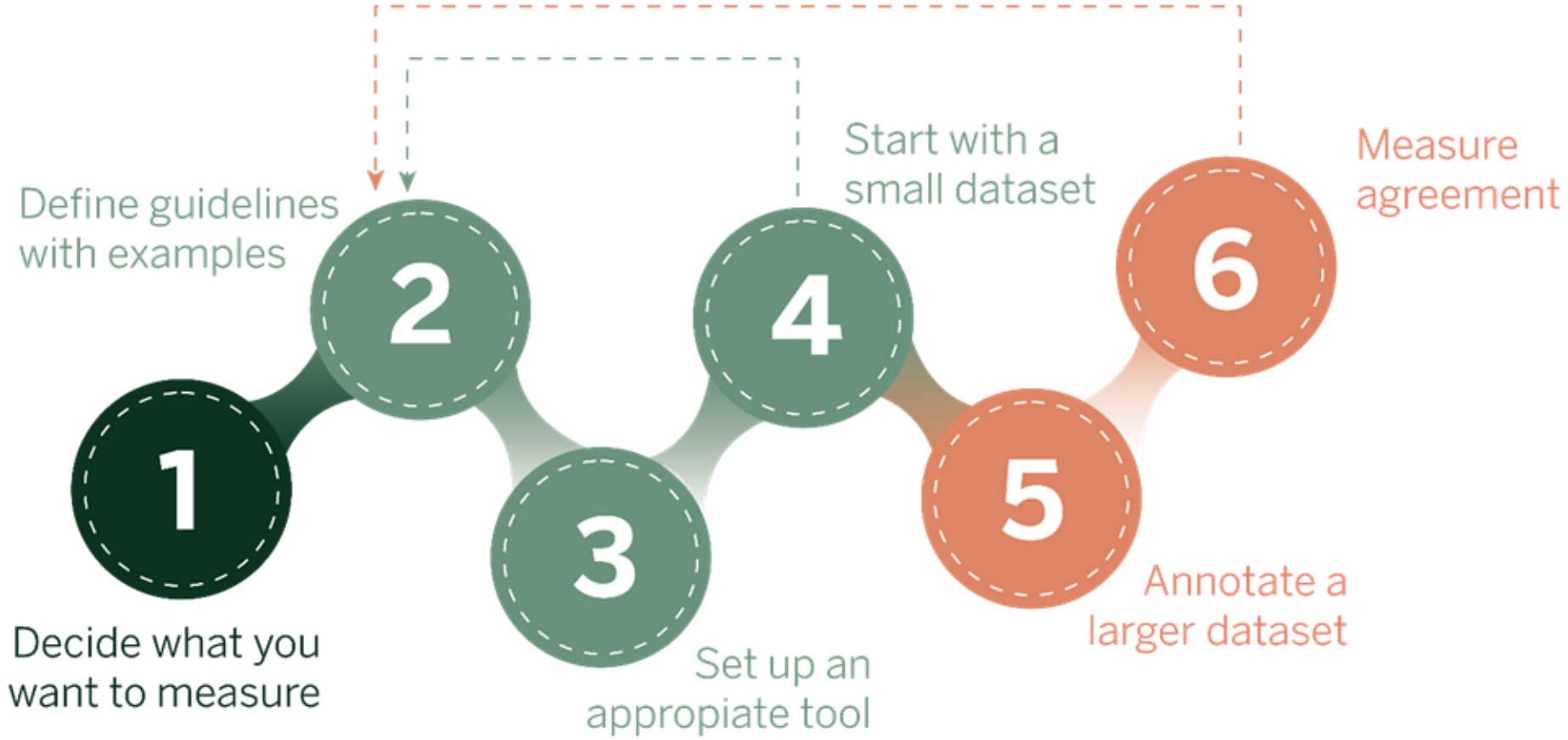
Krippendorff's Alpha

$$\alpha = 1 - \frac{D_o}{D_e}$$

D_o = observed disagreement
D_e = disagreement attributable to chance

- Any number of annotators
- Any number of categories
- Any level of measurement (nominal, ordinal, interval, etc.)
- Can be calculated on incomplete or missing data
- Cons: math gets more complicated, harder to interpret
- Online calculators or python packages (e.g., simpledorff)

The process is iterative!



Disagreement: noise or signal?

The usual approach

- Several people label each post → the **majority vote** becomes the “gold” label

The problem

- Hate speech is perceived differently, e.g. by people who are targeted by it
- The majority vote can **hide minority perspectives**
- Alternatives: keep all annotations, model each annotator, report the spread of labels

For your projects

- Report who annotated, how many annotators per item, agreement, and how disagreements were resolved

Davani, A. M., Díaz, M. & Prabhakaran, V. (2022). Dealing with Disagreements: Looking Beyond the Majority Vote in SubjectiveAnnotations. TACL 10.

Your turn: annotation exercise

Annotation exercise

Groups of three: annotators A, B, C, plus an AI as the final annotator

1. **Annotate alone in the shared sheet:** label the 10 posts with the categories from the reading
2. **Compare in your group:** count how often you agree
3. **AI as final annotator:** give the same definition and posts to an AI of your choice and enter its labels in the sheet
4. **Discussion**

Do not change your labels after comparing: disagreement is part of the result.

Step 1: the categories (from Vidgen & Derczynski 2020)

Label each post: P, G or N

- **P = person-directed abuse:** “Content which directs negativity against individuals, typically through aggression, insults, intimidation, hostility and trolling, amongst other tactics.”
- **G = group-directed abuse:** “Content which directs negativity against a social identity, which is defined in relation to a particular attribute (e.g. ethnic, racial, religious group membership).” It “includes use of dehumanising language, making derogatory, demonising or hostile statements, making threats, and inciting others to engage in violence, amongst other dangerous communications.”
- **N = neither**

Choose one label per post. If you think both P and G apply, pick the one you find stronger and make a note.

Vidgen, B. & Derczynski, L. (2020). Directions in abusive language training data, a systematic review: Garbage in, garbage out. PLOS ONE 15 (12): e0243300.

Step 3: prompt for the AI

You are annotating social media posts for a research project.
Categories: P = person-directed abuse: [paste definition]. G = group-directed abuse: [paste definition]. N = neither.
For each of the following 10 posts, answer with exactly one label (P, G or N) according to these definitions. Give one label per post and one short sentence explaining why.
Posts: [paste posts 1–10 with their numbers]

- Use exactly this prompt, so that we can compare the groups
- Do not show the AI your own labels

LLMs as annotators: what we know

Promising

- ChatGPT beat crowd workers (MTurk) on most of several annotation tasks, at a fraction of the cost (Gilardi et al. 2023)
- Fast, cheap, available in many languages

But

- Results depend on the **prompt, model and model version**, and can change over time
- LLMs can carry **biases** from their training data, and refuse or soften labels for offensive content
- They are trained on human-annotated data themselves
- Always **validate** against a human-labelled sample: treat the LLM like any other classifier

Gilardi, F., Alizadeh, M. & Kubli, M. (2023). ChatGPT outperforms crowd workers for text-annotation tasks. PNAS 120 (30). More on zero-shot classification with LLMs next week (Plaza-del-Arco et al. 2023).

Wrap-up

Reading 1: Jurafsky & Martin, ch. 4 in a nutshell

- **Text classification:** assign a label to a text (spam, sentiment, authorship, language, …)
- **Supervised learning:** learn from texts that humans have already labelled
- **Logistic regression:** features × weights + bias → sigmoid → probability → decision
- **Learning:** find the weights that make the fewest mistakes on the training data (cross-entropy loss, gradient descent)
- **More than two classes:** multinomial logistic regression (softmax)
- **Evaluation:** confusion matrix, precision, recall, F1, test sets and cross-validation
- **Harms:** classifiers can censor, misrepresent groups, and work worse for some groups

Take-aways

- **Classification** = learning labels from (human-)labelled examples
- Texts become **features** (e.g., bag of words), the model learns a **weight** per feature
- **Accuracy** is misleading with rare classes: use precision, recall and F1 (per class)
- Labels are made by people: **definitions, guidelines, annotators and sampling** shape everything the model learns
- Measure **agreement**, and take disagreement seriously
- AI annotators are tempting: they should be **validated** like any other model

Before next week (12 November)

- **Read:** Jurafsky & Martin, ch. 7 “Transformers and Pretraining” and Plaza-del-Arco, Nozza & Hovy (2023) “Respectful or Toxic? Using Zero-Shot Learning with Language Models to Detect Hate Speech”
- **Think about** your project: a phenomenon, a platform or a dataset, and a partner
- **19 November:** project idea (and group members) due

- All readings are linked in the syllabus on GitHub.

Universität Konstanz

Questions?
Thank you!

Jun.-Prof. Dr. Miriam Schirmer
Political and Administrative Science

miriam.schirmer@uni-konstanz.de

Credits: Parts of this slide deck have been adapted from slides created by Jurafsky & Martin, Debora Nozza, and Julia Mendelsohn.

Appendix

Dataset bias: an example

The Risk of Racial Bias in Hate Speech Detection (Sap et al. 2019)

- Tweets written in **African American English** were much more often labelled offensive in two widely used datasets
- Classifiers trained on these datasets learned this bias and flagged such tweets more often
- When annotators were told about the dialect and likely background of the author, they labelled fewer of these tweets as offensive

Lesson

- Biased labels → biased classifier → the people a moderation system should protect are censored more

Sap, M., Card, D., Gabriel, S., Choi, Y. & Smith, N. A. (2019). The Risk of Racial Bias in Hate Speech Detection. ACL.

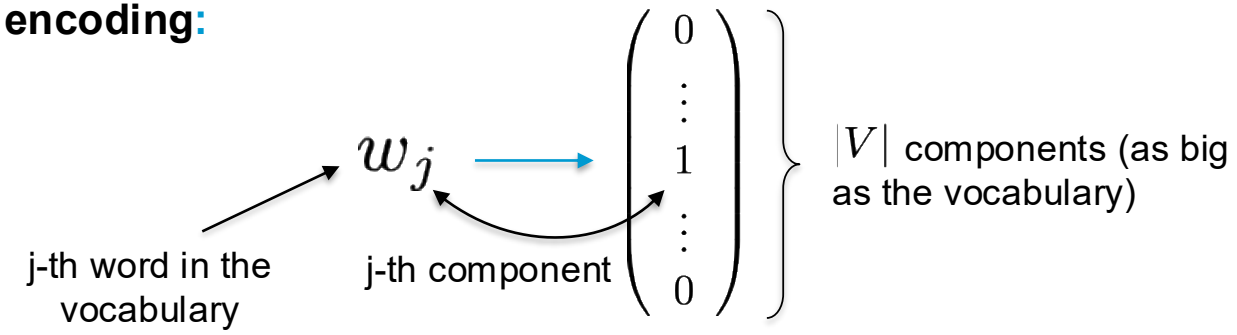
Bag of words (BoW)

One-hot encoding of words

Complementary point of view: map words into vectors:

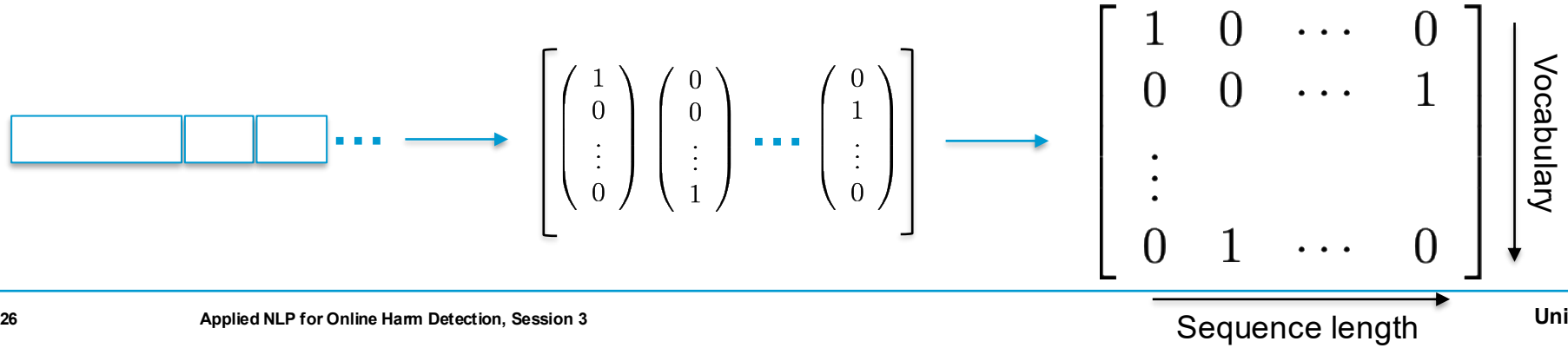


One-hot encoding:



Matrix representation for a document

A document is mapped to a sequence of one-hot vectors, which can be stacked into a matrix:



Binary logistic regression for one observation *x*

Input: vector $x = [x_1, x_2, \ldots, x_n]$

Weights: one per feature: $W = [w_1, w_2, \ldots, w_n]$

- Sum up all the weighted features bias term b

$$z = \left(\sum_{i=1}^{n} w_i x_i\right) + b$$

$$z = w \cdot x + b$$

Output: predicted class $\hat{y} \in \{0, 1\}$ based on the sum z

- If the sum is high, we predict $\hat{y} = 1$
- If the sum is low, we predict $\hat{y} = 0$
- We use the *sigmoid* function to convert z to a probability from 0 to 1

$$y = \sigma(z) = \frac{1}{1 + e^{-z}}$$

$$\hat{y} = \begin{cases} 1 & \text{if } P(y = 1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

Sentiment Example

Suppose w = [2.5, -5.0, -1.2, 0.5, 2.0, 0.7] and **b = 0.1**

And we extracted the feature vector, **x = [3, 2, 1, 0, 4.19]**

$$\begin{aligned} p(+|x) = P(Y=1|x) &= \sigma(w \cdot x + b) \\ &= \sigma([2.5, -5.0, -1.2, 0.5, 2.0, 0.7] \cdot [3, 2, 1, 3, 0, 4.19] + 0.1) \\ &= \sigma(.833) \\ &= 0.70 \end{aligned} \tag{5.6}$$

$$\begin{aligned} p(-|x) = P(Y=0|x) &= 1 - \sigma(w \cdot x + b) \\ &= 0.30 \end{aligned}$$

Cross-entropy Loss

Goal: maximize the probability of the correct label p(y|x)

With a binary classifier, the probability from our classifier, and the thing we want to maximize is:

- If y = 1, this is just ŷ,
- If y = 0 this is just 1-ŷ

$$p(y|x) = \hat{y}^y (1-\hat{y})^{1-y}$$

Because of math, it's easier to take the log and flip the sign to minimize:

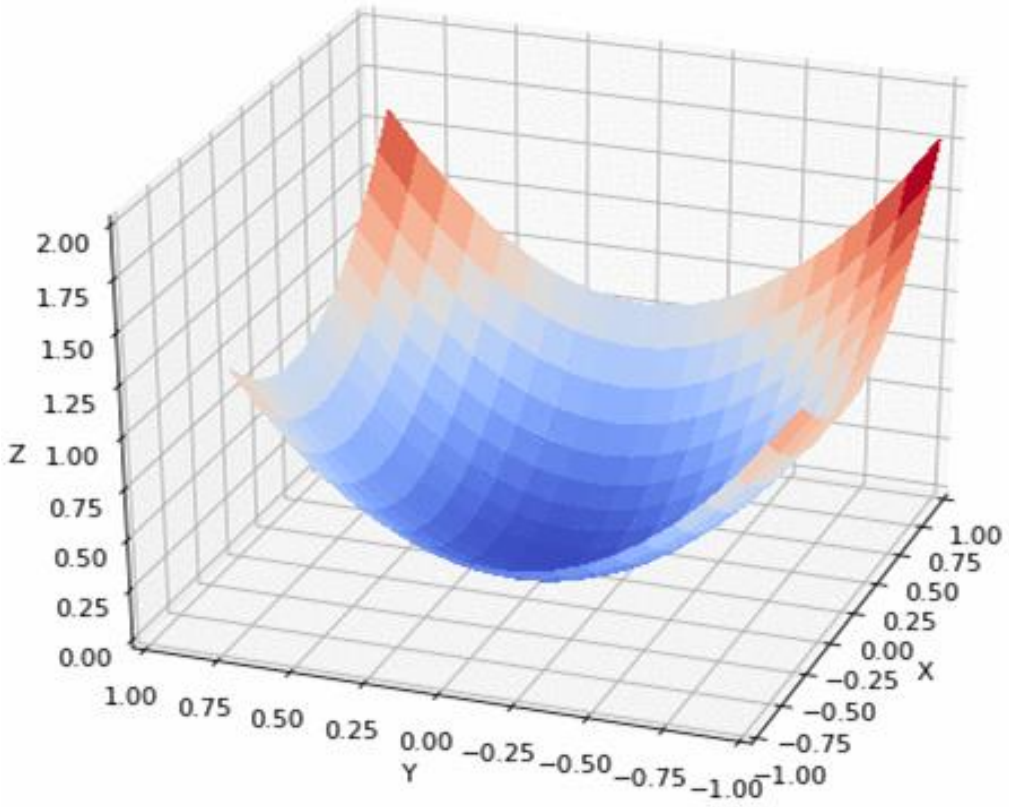
$$L_{\text{CE}}(\hat{y}, y) = -[y \log \sigma(w \cdot x + b) + (1-y) \log (1 - \sigma(w \cdot x + b))]$$

ŷ

ŷ

Gradient Descent

- The gradient of a function is a vector pointing in the direction of greatest increase
- **Gradient descent**: Find the gradient of the loss function at the current point and move in the opposite direction
  - Gradient descent calculates the gradient from the entire training dataset → *very slow*
  - **Stochastic gradient descent** is more widely used (update after a small batch of samples)
- The **learning rate** hyperparameter governs how much you move in that direction
  - Too high: you overshoot the minimum
  - Too low: super slow



Turning a probability into a classifier

$$\hat{y} = \begin{cases} 1 & \text{if } P(y=1|x) > 0.5 \\ 0 & \text{otherwise} \end{cases}$$

0.5 here is called the **decision boundary**

Intuition of negative log likelihood loss = cross-entropy loss

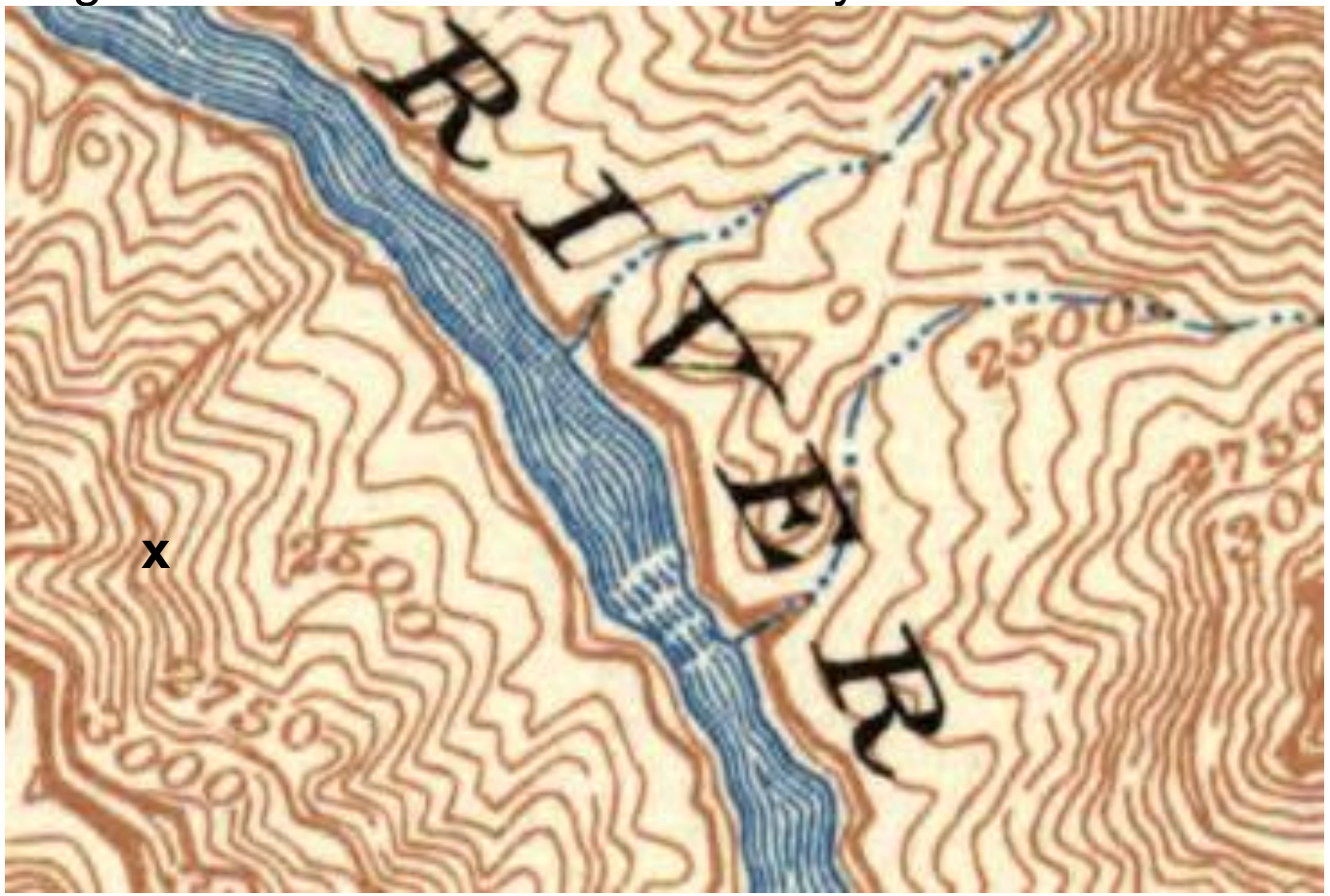
A case of conditional maximum likelihood estimation

We choose the parameters w,b that maximize

- the log probability
- of the true y labels in the training data
- given the observations x

Intuition of gradient descent

How do I get to the bottom of this river canyon?



Look around me 360°

Find the direction of steepest slope down

Go that way

F1 is a combination of precision and recall.

A combined measure: F1

$$F_1 = \frac{2PR}{P + R}$$

How to combine P/R values for different classes: Microaveraging vs Macroaveraging

Class 1: Urgent

	true urgent	true not
system urgent	8	11
system not	8	340

$$\text{precision} = \frac{8}{8+11} = .42$$

Class 2: Normal

	true normal	true not
system normal	60	55
system not	40	212

$$\text{precision} = \frac{60}{60+55} = .52$$

Class 3: Spam

	true spam	true not
system spam	200	33
system not	51	83

$$\text{precision} = \frac{200}{200+33} = .86$$

$$\text{macroaverage precision} = \frac{.42+.52+.86}{3} = \mathbf{.60}$$

Pooled

	true yes	true no
system yes	268	99
system no	99	635

$$\text{microaverage precision} = \frac{268}{268+99} = \mathbf{.73}$$

Beyond one F1 score

A single score on one test set can hide a lot

- **Other datasets:** models trained on one hate speech dataset often perform much worse on another
- **Functional tests:** HateCheck tests models on targeted cases, e.g. negation, counter-speech, reclaimed slurs, spelling variations (Röttger et al. 2021)
- **Errors per group:** are posts mentioning certain identities, or written in certain dialects, flagged more often? (Dixon et al. 2018; Sap et al. 2019)
- **Look at the errors:** read false positives and false negatives yourself

Röttger, P. et al. (2021). HateCheck: Functional Tests for Hate Speech Detection Models. ACL. Dixon, L. et al. (2018). Measuring and Mitigating Unintended Bias in Text Classification. AIES. Sap, M. et al. (2019). The Risk of Racial Bias in Hate Speech Detection. ACL.

1. Decide what you want to measure

- What is the **construct** you're interested in, and what are the **categories** you want to label?
- What is your **unit of analysis**? What are the chunks for each annotation task? (Text spans? Sentences? News articles? Social media posts? Entire threads?)
- What is your **sample?** What data will you annotate?
  - Consider where the model will be used, and select data representative of use cases
  - Will models trained on news articles generalize to Reddit data?
  - Access and appropriate permission for the ideal data?



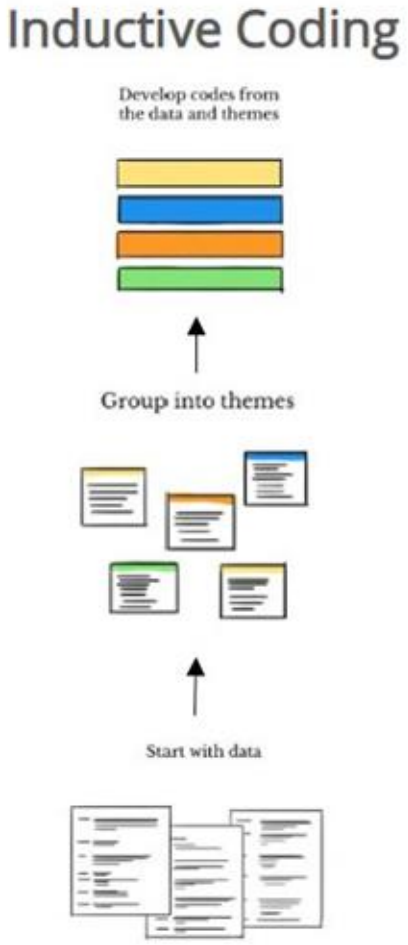
2. Define guidelines with examples

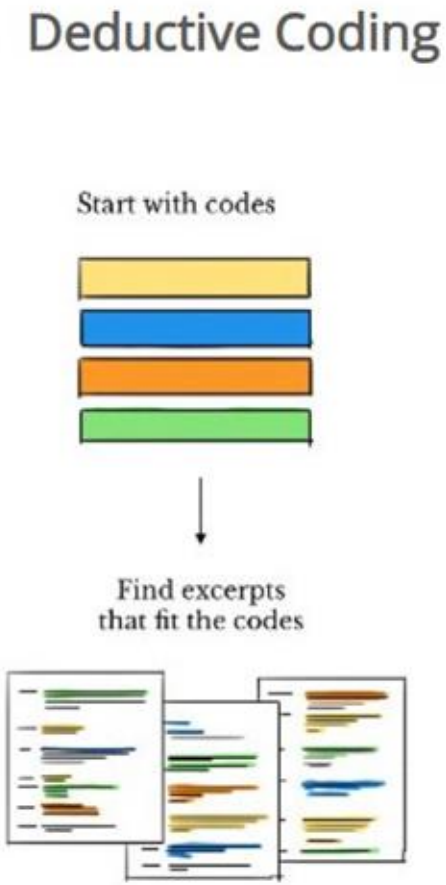
- A **codebook** contains the list of categories you want to annotate for ("codes") and descriptions
- The ideal codebook is **comprehensive but concise**. Clear definitions and examples of what should be included or excluded under each code are helpful.
- How many categories should a codebook contain?
  - Depends on your research question, what claims you want to be able to make, and who your annotators are
  - Ability to do fine-grained analysis is great but too many categories makes annotation a nightmare
  - When in doubt, start with more categories and merge later.



Inductive and deductive coding

- **Deductive coding** (top-down): use pre-defined scheme, e.g. from existing literature
  - e.g. Plutchik or Ekman's emotion taxonomies
- **Inductive coding** (bottom-up): infer labels by looking at your data
  - e.g. media frames (Boydstun, 2014)
  - Can inform future works' deductive coding
- Inductive and deductive coding can be used together





3. Set up an appropriate tool

- You want an annotation interface that fits your needs and makes the actual process as easy as possible
  - E.g. specific tools for highlighting spans of text or relations
- Commonly-used tools:
  - LabelStudio, Prodigy, Qualtrics, Doccano, custom websites
  - Prodigy is $$$ but apparently very nice and gives free licenses to academics if you email with your university details
- My most commonly-used: Google Sheets
  - I use drop down boxes/chips to avoid typos
  - And color code (conditional formatting)



4. Start with a small dataset

- Analogous to conducting pilot studies
- Sample a little bit of data to make sure the categories make sense and are identifiable in your corpus
- Adjust codebook to be a more useful and reliable tool
  - Remove categories that don't make sense given your data
  - Add categories that seem relevant and important to answer your research questions that may not be in the codebook
  - Adjust inclusion/exclusion criteria based on your observations
  - Include a couple examples into the codebook



4. Start with a small dataset

My workflow for training annotators (e.g. undergrad RAs)

1. Start with initial codebook
2. All annotators independently label a small sample
3. Calculate agreement
4. Meet to resolve disagreements via **consensus-coding**
5. Update codebook to address source of disagreement
6. Repeat with new small samples until high agreement

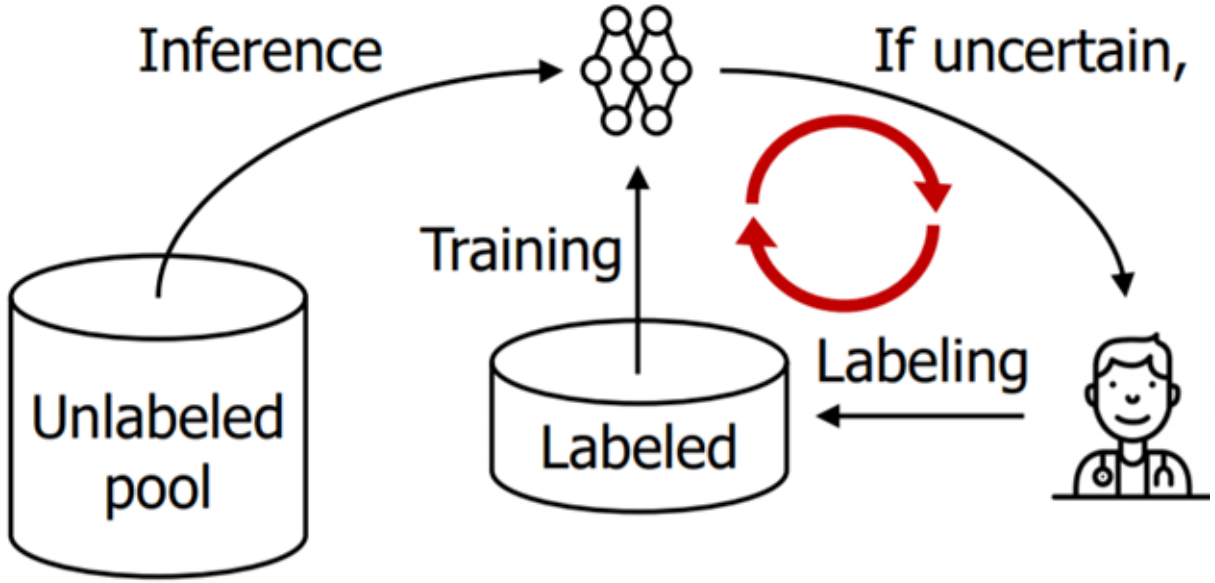


How do you choose data to annotate?

- Option 1: **randomly sample data**
- **Advantages of random samples**
  - Representative sample gives unbiased estimates, e.g. of category prevalence
  - Captures natural variation in the data, and annotation reliability measures would more clearly reflect “real world” performance
- Disadvantages of random samples
  - Some constructs are rare and may not appear much in a random sample.
  - E.g. abusive tweets are only 0.1-3% of the data (depending on label) [Founta et al., 2018]
  - If we don’t have enough annotations for some categories, our models won’t work well

How do you choose data to annotate?

- Option 1: randomly sample data
- Option 2: pre-filtering
- Option 3: **active learning**
  - Use whatever labeled data you have to train a classifier.
  - Run classifier on unlabeled data to identify samples with high uncertainty.
  - Manually annotate those samples and repeat until you have enough data



https://smilegate.ai/2021/06/23/learning-loss-for-active-learning/

- Advantage: selects samples that yield most improvement for modeling
- Disadvantage: introduces model-driven biases

5. Annotate a larger dataset

Once you're confident in your annotation scheme and codebook, you can annotate the larger dataset you'll use to train and evaluate models. A few options:

- Have multiple trained annotators label all samples
  - You can measure agreement periodically for calibration and consensus-code to resolve all disagreements
- Divide remaining data among now-trained annotators
  - You might want to still calibrate by having everyone do the same small set periodically (especially for long-running tasks)
- If the task is simple and the codebook is short, you can use crowdsourcing.
  - Get multiple annotations per item as your budget allows



Crowdsourcing considerations

- Platforms:
  - Prolific
  - Amazon Mechanical Turk
  - CloudResearch
  - Toloka
- Quality control
  - Annotator qualifications (# tasks done, % accepted, demographics/interests if applicable)
  - “Gold tasks”/attention checks (very basic questions to make sure they’re paying attention)
  - Try to minimize annotators’ LLM use by (1) explicitly asking them not to use LLMs and (2) disabling copy/paste in open-ended responses
  - Release data in small batches so you don’t waste $1000s on bad data
  - Pay fairly!! (determine estimated time in pilot tests, refine based on crowdsourced batches)
- Especially for crowdsourcing, give each item a unique identifier

Cohen's Kappa

		Annotator 1		
		Not offensive	**Offensive**	**Sum**
Annotator 2	**Not offensive**	**147**	3	150
	Offensive	10	**62**	72
	Sum	157	65	222

$$\kappa = \frac{p_o - p_e}{1 - p_e}$$

p_o = percent agreement

p_e = chance agreement

0 → agreement is random chance

\- → agreement is worse than random

Interpretation of Kappa

	Poor	Slight	Fair	Moderate	Substantial	Almost perfect
Kappa	0.0	.20	.40	.60	.80	1.0

Kappa	Agreement
< 0	Less than chance agreement
0.01–0.20	Slight agreement
0.21– 0.40	Fair agreement
0.41–0.60	Moderate agreement
0.61–0.80	Substantial agreement
0.81–0.99	Almost perfect agreement